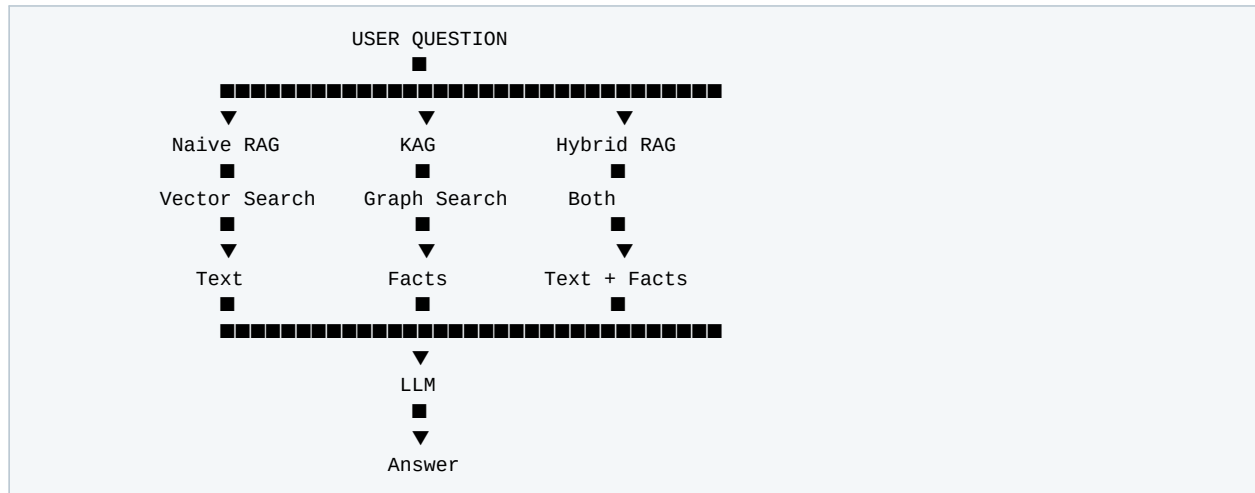


RAG, KAG and Hybrid RAG

A simple guide to the three retrieval approaches used in this project



The core idea

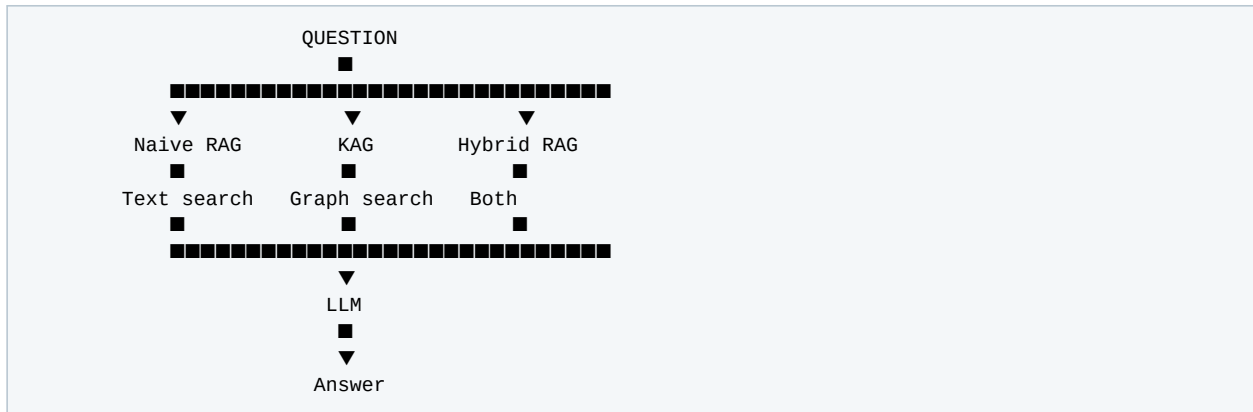
Naive RAG searches for relevant passages. KAG searches for structured facts and relationships. Hybrid RAG combines both.

This document explains the architecture, strengths, limitations and evaluation of the three approaches without assuming prior knowledge of RAG or Knowledge Graphs.

1. Overview

The project demonstrates three ways of answering questions from a document:

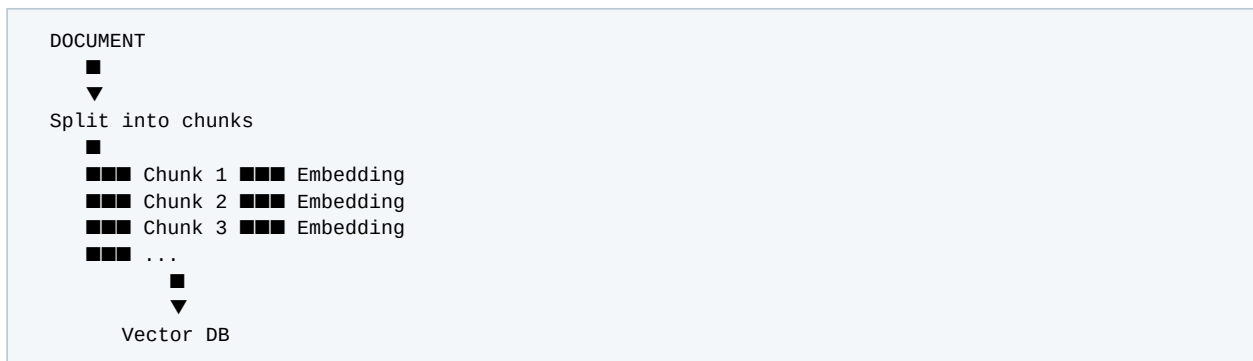
- Naive RAG — finds relevant text using vector similarity.
- KAG — finds answers by traversing a Knowledge Graph.
- Hybrid RAG — combines vector retrieval, graph facts and complete entity sets.



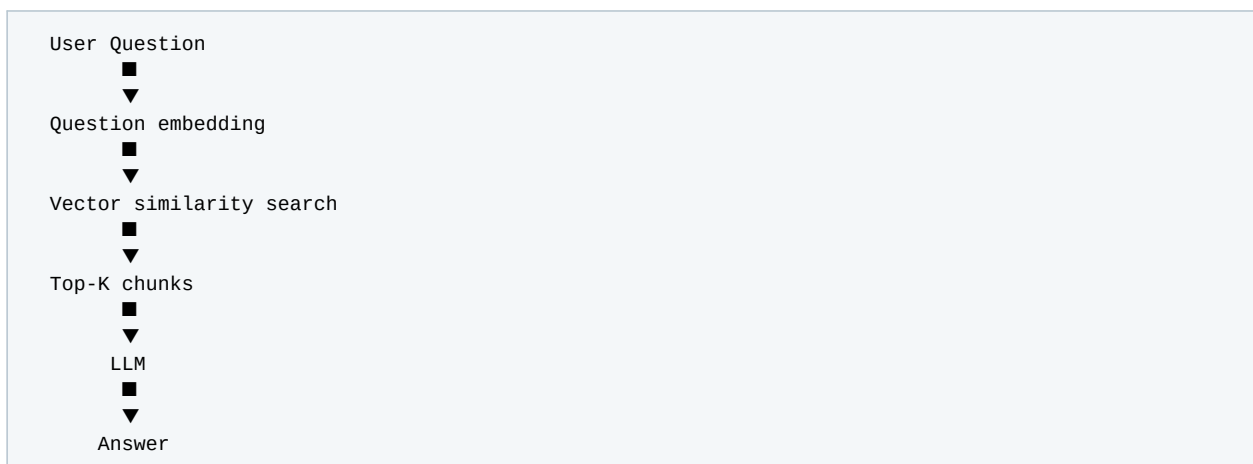
A useful mental model is: text is best for wording and prose; graphs are best for relationships and exhaustive structured questions.

2. Naive RAG

Naive RAG is the simplest approach. The PDF is divided into smaller text chunks. Each chunk is converted into an embedding, stored in a vector database, and later retrieved by similarity to the user's question.



At query time:



The project uses pypdf for PDF extraction and a recursive text splitter with a chunk size of 700 characters and 100 characters of overlap. Each chunk receives a stable ID such as chunk_p3_2.

Embeddings use BAAI/bge-small-en-v1.5. The vectors are stored in Chroma using a cosine-based HNSW index. Queries use the model's search prefix before embedding the question.

3. The Main Limitation of Naive RAG

The key limitation is simple: top-k retrieval is a sample, not a census.

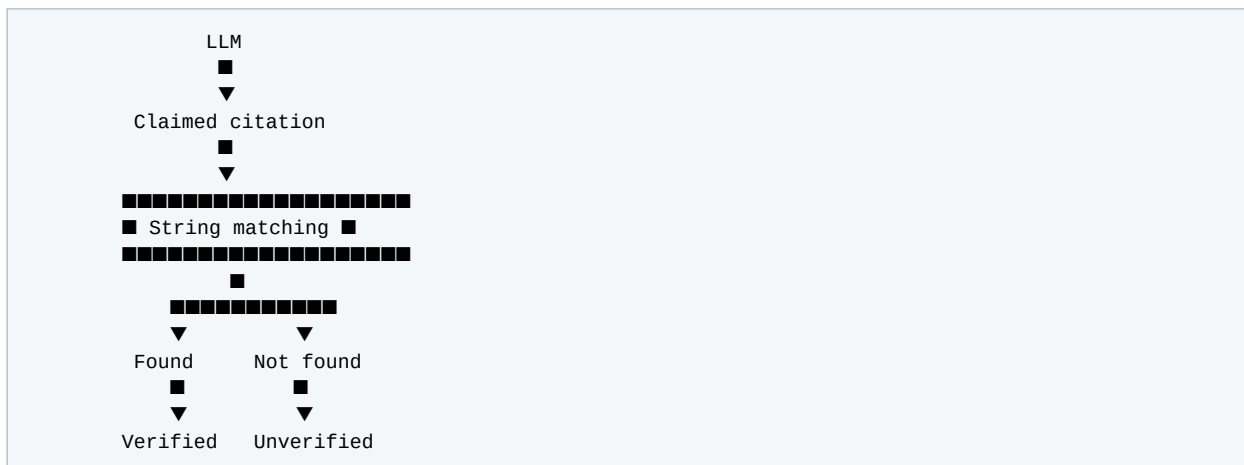


This is fine for a local question such as “What entrance exam does B.Des require?”, because the answer may be present in one or two chunks.

It is much weaker for global questions such as “How many distinct entrance examinations are mentioned?” or “List all schools.” Increasing k helps, but no practical k guarantees that every relevant item will be retrieved.

4. Citation Verification in Naive RAG

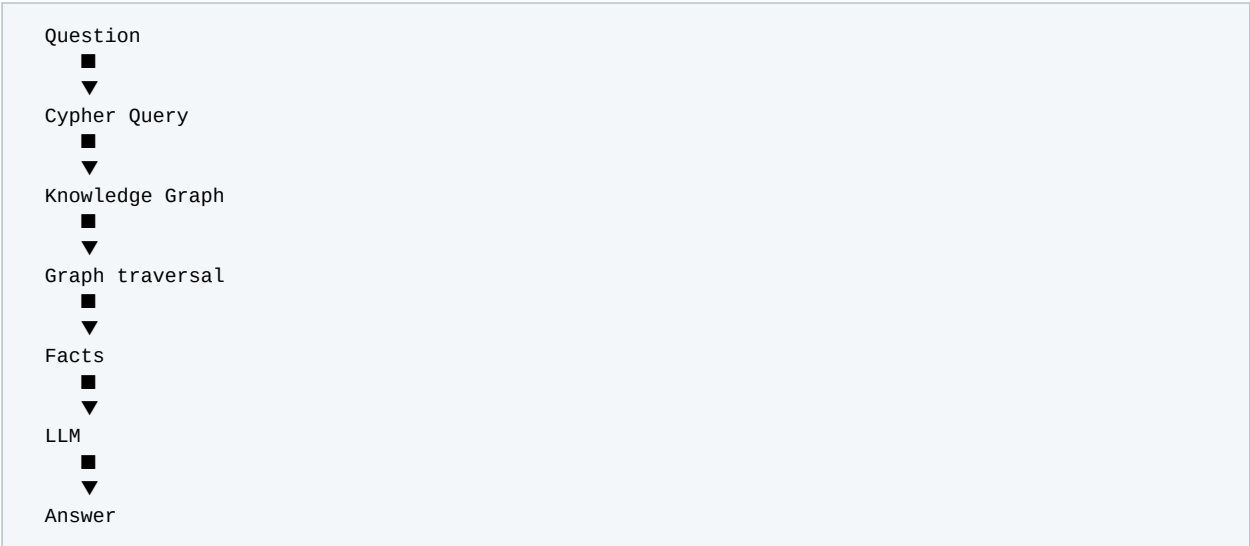
The LLM is required to return a SOURCES block containing a chunk ID and a verbatim sentence. The system then checks whether the claimed quote actually occurs in that chunk.



This is a string check, not a second model judging the citation. Unverified claims are reported rather than silently accepted.

5. KAG — Knowledge-Augmented Generation

KAG takes a different approach. Instead of searching for similar text, it answers questions using entities and relationships stored in a Knowledge Graph.



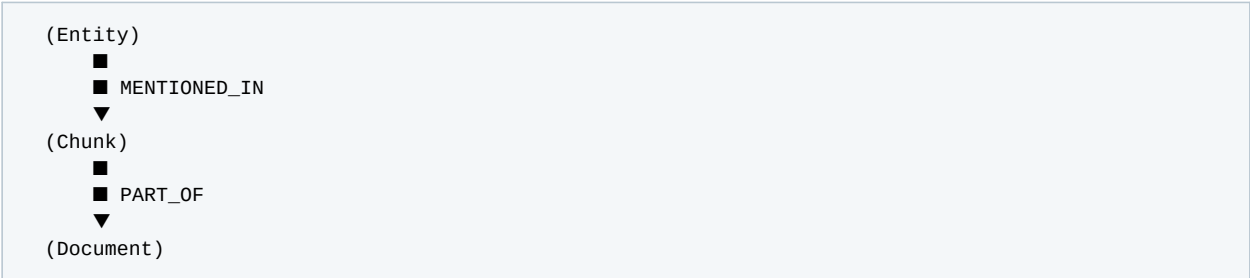
The graph is stored in Neo4j. The ontology defines entity types such as Programme, School, Facility, EntranceExam, Regulation, AdmissionCriterion and others.

Relationships describe how entities are connected. For example:

```
School ■■■OFFERS■■■■■■■■■■■ Programme
School ■■■HAS_FACILITY■■■■■ Facility
Programme ■■■REQUIRES■■■■■ EntranceExam
```

6. Connecting the Graph Back to the Document

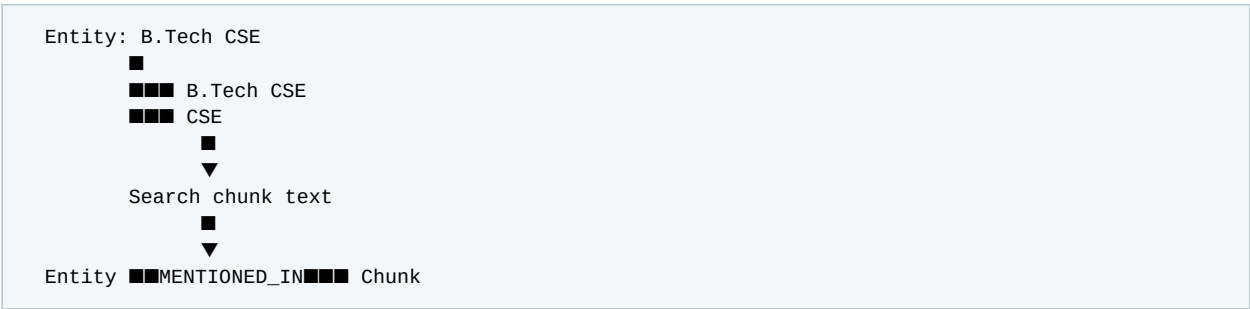
A major strength of this implementation is that graph entities are connected back to the original text chunks.



This allows the system to provide provenance: a graph-derived answer can still be traced back to the source passage and page.

7. Entity Linking

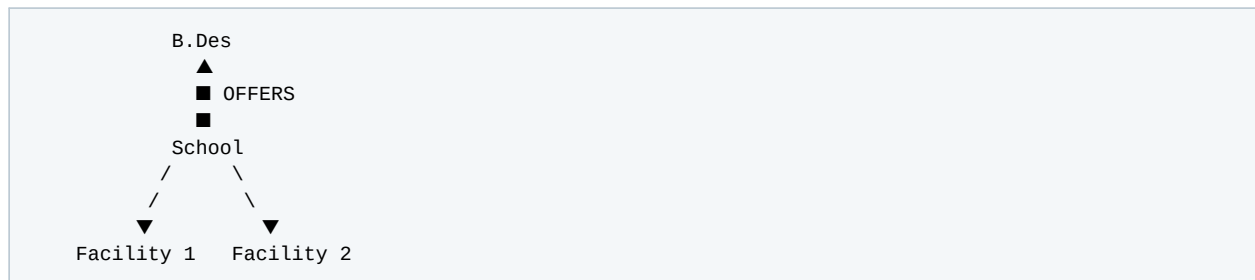
Entity linking is deterministic in this project; it does not require an LLM. Names, aliases and derived forms are matched against chunk text. For example, B.Tech CSE can also register CSE as a searchable form.



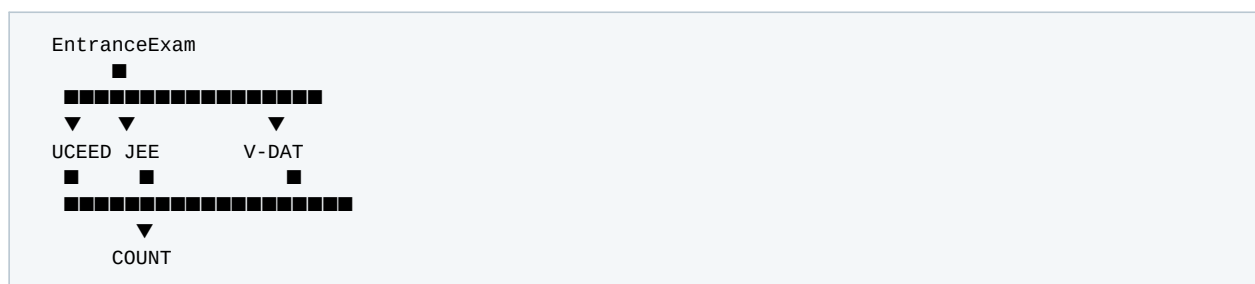
An orphan report identifies entities that have no linked chunk, providing a useful quality check on the linker.

8. What KAG Can Do Better

Multi-hop questions: a question can require several relationships to be followed.

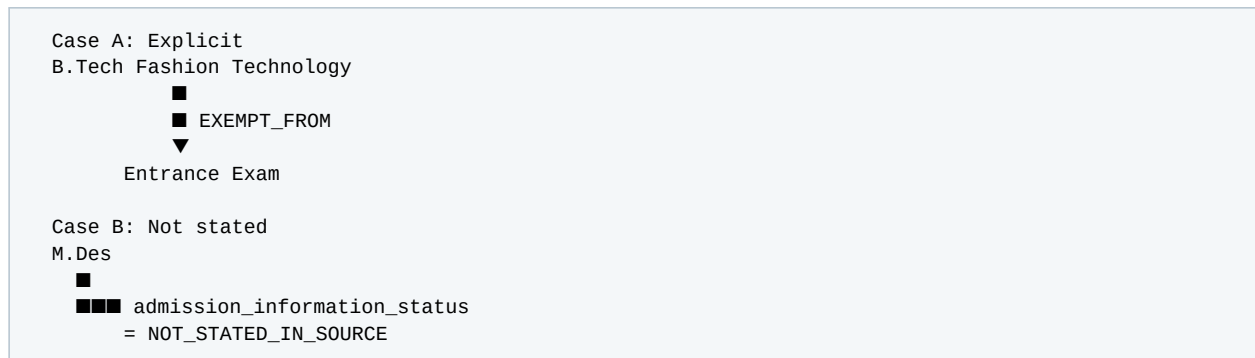


Global questions: the graph can inspect all nodes of a type rather than relying on a small retrieved sample.



9. Explicit Negation and Known vs. Unknown

The graph can distinguish between an explicit statement and missing information. For example, an explicit EXEMPT_FROM relationship means the source says a programme does not require an entrance exam. A NOT_STATED_IN_SOURCE status means the source does not provide enough information.



This prevents the system from treating “the source says there is no exam” and “the source does not tell us” as the same thing.

10. KAG's Main Limitation

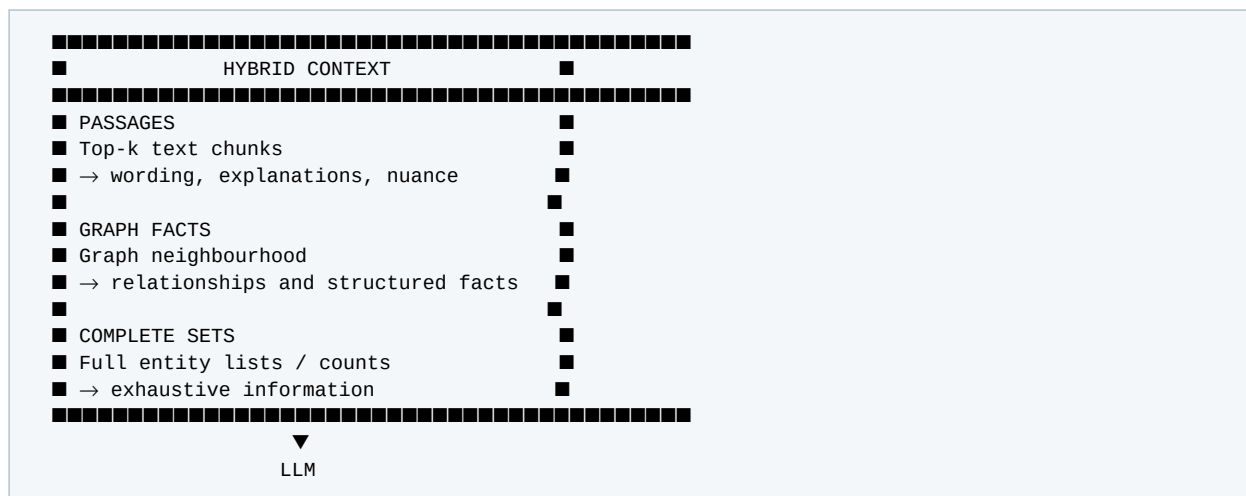
The Knowledge Graph is a structured representation of the document, not the complete document itself. Information that was never modelled as entities or relationships cannot be recovered by graph traversal.



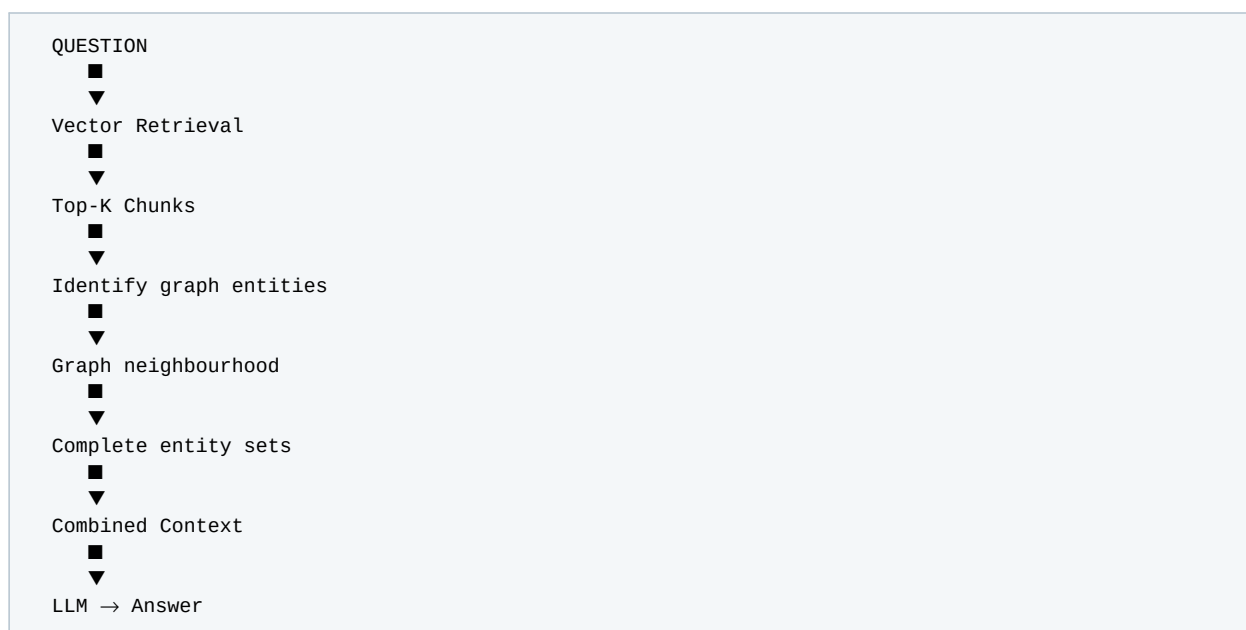
This is the lossy projection problem. A qualitative question such as “What do employers expect from students?” may require the original prose rather than graph facts.

11. Hybrid RAG

Hybrid RAG combines the strengths of both approaches. Its context has three blocks: passages, graph facts and complete sets.



12. Hybrid RAG Query Flow



The implementation first retrieves four chunks. It then seeds the graph using both the retrieved chunks and the question itself.

13. Dual Graph Seeding

This is one of the most important design decisions in the hybrid implementation.

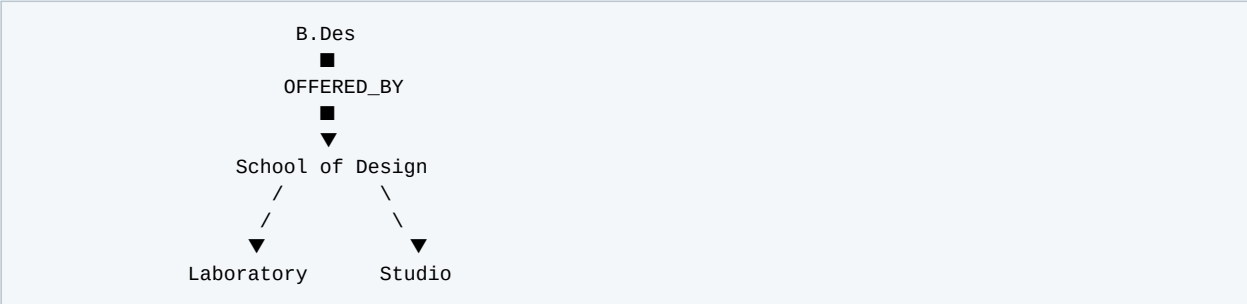


For example, the word “schools” can seed the School label, while “facilities”, “labs” or “studios” can seed the Facility label.

This prevents Hybrid RAG from inheriting every vector-retrieval failure. Even if the top retrieved chunks do not mention a school, the question itself can still ask the graph for School entities.

14. Graph Neighbourhood Expansion

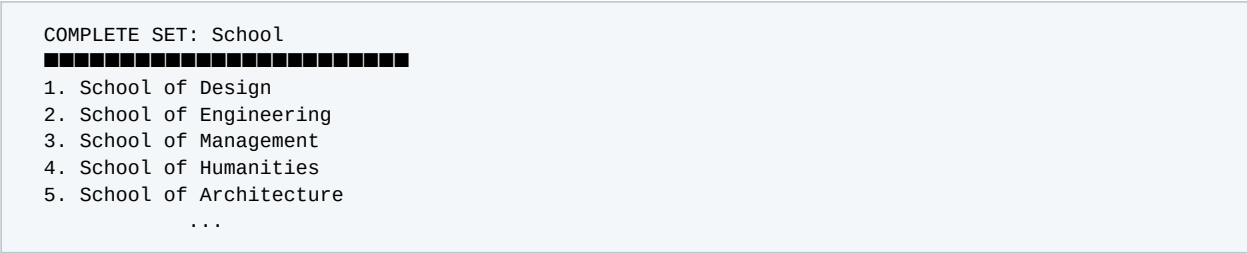
Once an entity is found, Hybrid RAG expands to its neighbours and retains relationship properties.



Properties such as evidence, condition_group and source_note are retained because they can carry the exact detail needed to justify an answer.

15. Complete Sets

For questions such as “List all schools” or “How many entrance examinations are there?”, Hybrid RAG provides a complete set of the relevant graph entity type.



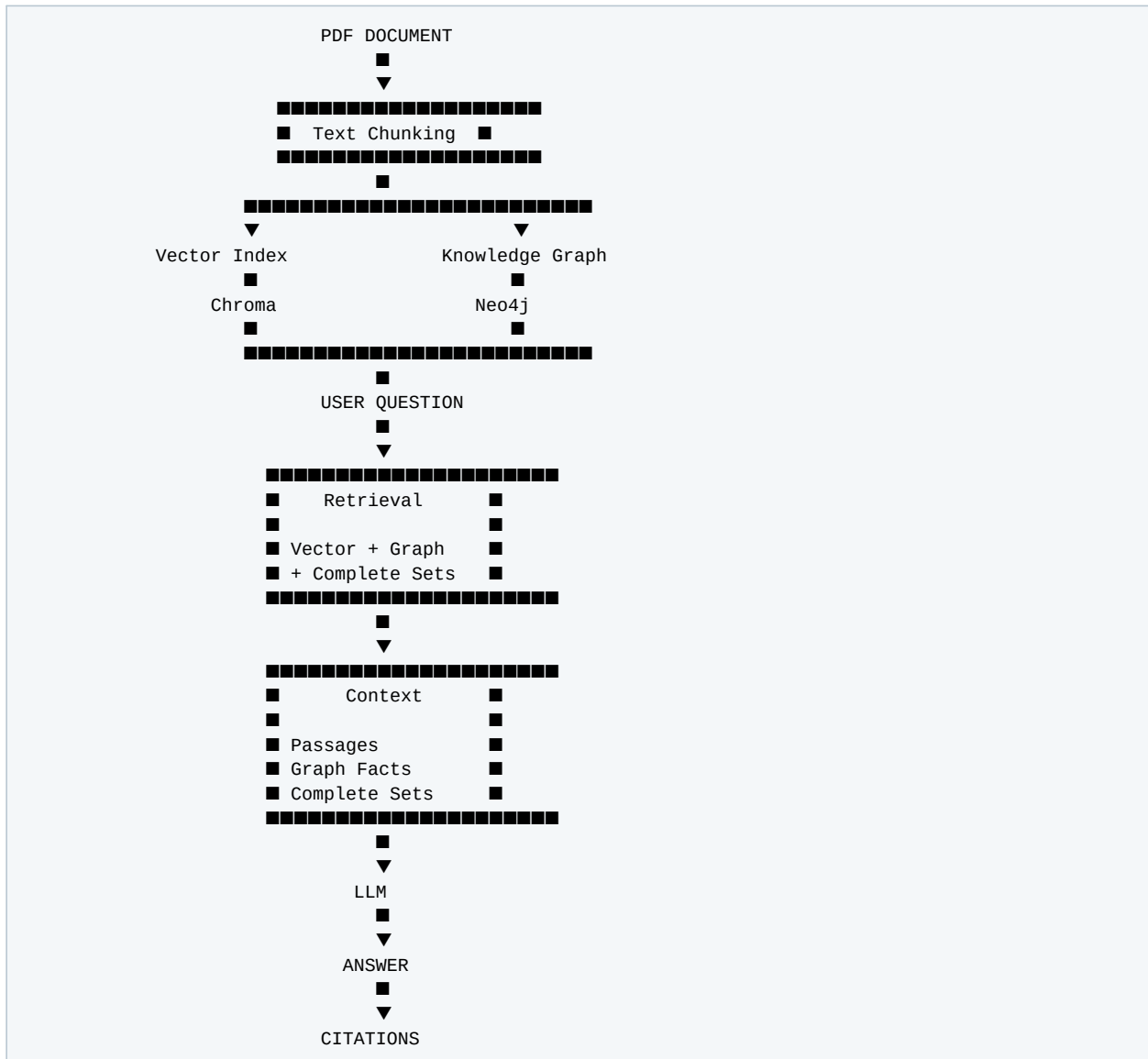
The prompt tells the LLM to trust COMPLETE SETS over passages when the question asks “how many” or “list all”. This is important because passages are samples, while complete sets are intended to be exhaustive.

16. Comparing the Three Approaches

Capability	Naive RAG	KAG	Hybrid RAG
Text retrieval	Yes	Limited	Yes
Natural-language prose	Strong	Limited	Strong
Multi-hop relationships	Weak	Strong	Strong

Approach	Local	Global	Prose
Passages	Strong	Weak	Strong
Graph	Strong	Strong	Weak
Hybrid	Strong	Strong	Strong

20. End-to-End Architecture



The architecture makes the division of responsibility clear: the vector store preserves the richness of the original text; the graph adds explicit structure and relationships; Hybrid RAG brings both forms of evidence together.

21. Key Takeaways

Naive RAG — Search the text.

Simple and effective when the answer is contained in a few relevant passages. Its weakness is that top-k retrieval cannot guarantee complete coverage.

KAG — Traverse the knowledge.

Excellent for relationships, multi-hop questions, counts, explicit negation and structured facts. Its weakness is that the graph only contains what was deliberately modelled.

Hybrid RAG — Use both.

Combines relevant passages, graph relationships, complete entity sets, provenance and citation verification.

Naïve RAG	■■■■■■■■■■	Text / prose / local evidence
KAG	■■■■■■■■■■	Structure / relationships / global facts
Hybrid	■■■■■■■■■■	Text + structure + complete sets

Final principle: Do not treat every question as a vector-search problem. Questions such as “What does the document say?” may need text, while “How many?”, “List all”, and relationship questions may need graph traversal. Many real-world questions need both.